#SKILL - 9 Global Exception Handling using @ControllerAdvice

Prerequisites:

- Understanding of REST API exception flow
- Concept of custom exceptions

A university application provides student details through REST APIs. When users enter an invalid student ID, the system shows technical error messages that are difficult for users to understand. To improve usability, the application should return clear, readable, and userfriendly error responses through centralized exception handling.

Your task is to implement Global Exception Handling using @ControllerAdvice and create custom exceptions for invalid student inputs.

Tasks:

1. Create a custom exception class named StudentNotFoundException.

```java
package com.example.student.exception;

public class StudentNotFoundException extends RuntimeException {

    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

2. Create a REST endpoint /student/{id} that retrieves student information and throws the custom exception when the given student ID is invalid.

```java
package com.example.student.controller;

import org.springframework.beans.factory.annotation.Autowired;

@RestController
@RequestMapping("/student")
public class StudentController {

    @Autowired
    private StudentService studentService;

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable String id) {

        int studentId;

        try {
            studentId = Integer.parseInt(id);
        } catch (NumberFormatException e) {
            throw new InvalidInputException("Student ID must be numeric");
        }

        return studentService.getStudentById(studentId);
    }
}
```

3. Create a GlobalExceptionHandler class using @ControllerAdvice to handle exceptions across the entire application.

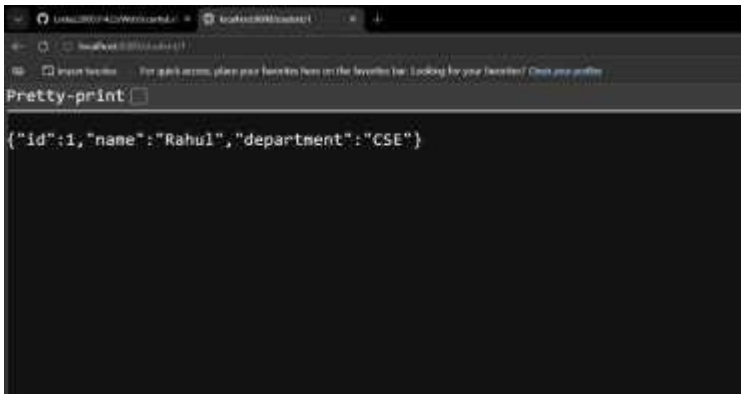
```java
package com.example.student.exception;

import java.time.LocalDateTime;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleStudentNotFound(StudentNotFoundException ex){

        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(),
                ex.getMessage(),
                HttpStatus.NOT_FOUND.value()
        );

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ErrorResponse> handleInvalidInput(InvalidInputException ex){

        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(),
                ex.getMessage(),
                HttpStatus.BAD_REQUEST.value()
        );

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}
```

4. Add a method with @ExceptionHandler(StudentNotFoundException.class) to return a meaningful error response along with an appropriate HTTP status code.

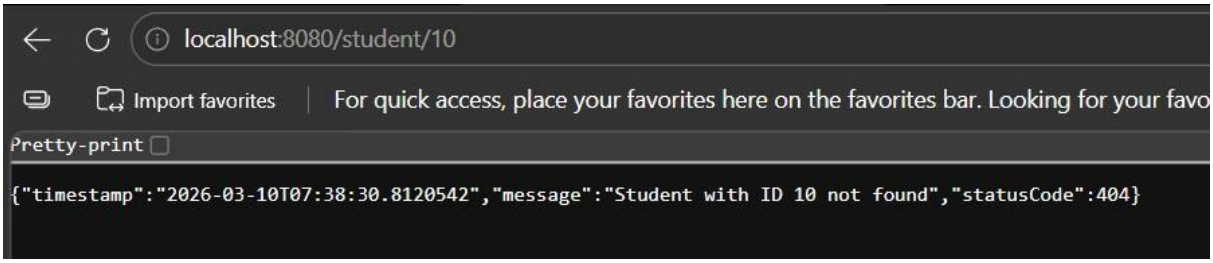
5. Add a second custom exception, such as InvalidInputException, and handle it inside the same @ControllerAdvice class.

```java
package com.example.student.exception;

public class InvalidInputException extends RuntimeException {

    public InvalidInputException(String message) {
        super(message);
    }
}
```

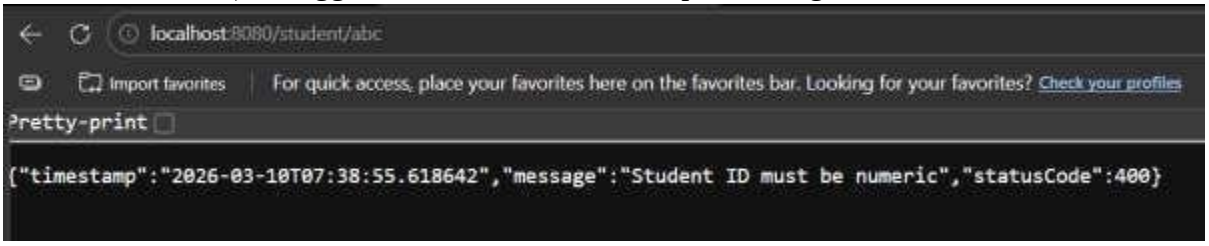
6. Return a structured JSON response (fields like timestamp, message, statusCode) from the exception handler to make the error readable and consistent.



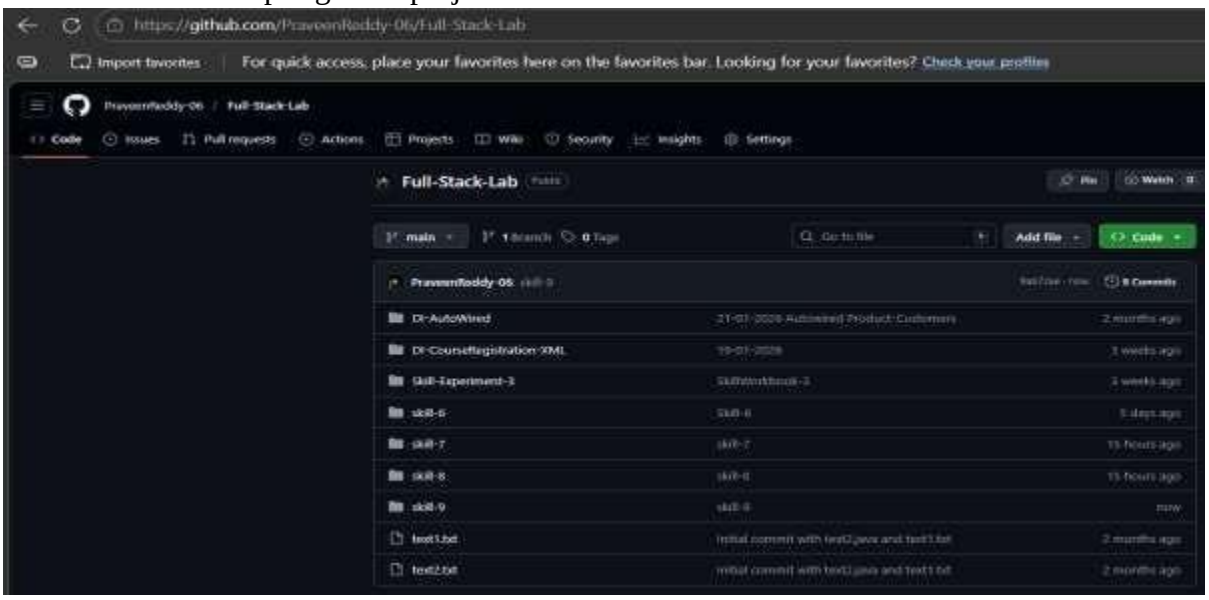
7. Test the API with valid and invalid student IDs and observe the difference in responses.



8. Test an endpoint with invalid input format (e.g., sending text instead of a number) to trigger the second custom exception using Postman.



9. Push the Spring Boot project to GitHub.



GitHub Link: https://github.com/Anil1843/fsad-skill

VIVA QUESTIONS:

1. **What is the purpose of @ControllerAdvice in Spring?**
@ControllerAdvice is used to handle exceptions globally across all controllers in a Spring Boot application. It centralizes exception handling logic instead of writing try-catch blocks in every controller.

2. **How does @ExceptionHandler work inside a global handler?**
@ExceptionHandler defines methods that handle specific exception types. When an exception occurs, Spring automatically calls the matching handler method and returns a custom response.

3. **Why do applications use global exception handling?**
Global exception handling provides:
- centralized error handling • cleaner controller code
- consistent error responses
- better user-friendly API responses.

4. Difference between custom exceptions and default exceptions? Default exceptions are built-in Java exceptions (NullPointerException, IllegalArgumentException).
Custom exceptions are user-defined exceptions created to represent specific application errors such as StudentNotFoundException.

5. Can a single @ControllerAdvice class handle multiple exceptions?
Yes. A single @ControllerAdvice class can handle multiple exceptions by defining multiple @ExceptionHandler methods for different exception types.

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured: _____ out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation: